ECSE 429 Software Validation

Part C: Non-Functional Testing of REST API

Written Report

Emile Labrunie (261097953) – emile.labrunie@mail.mcgill.ca
Ethan Wu (261117309) – ethan.wu2@mail.mcgill.ca
Kenny Duy Nguyen (261120429) – kenny.nguyen@mail.mcgill.ca
Yanzhe Zhang (261016377) – yanzhe.zhang@mail.mcgill.ca

Winter 2026

Contents

1 Introduction

This report presents the non-functional testing performed on the REST API Todo List Manager application as Part C of the ECSE 429 term project. Building on the functional validation conducted in Parts A and B, this phase focuses on two complementary aspects of software quality: *performance testing* (dynamic analysis) and *static analysis* of the application source code.

The performance testing component measures transaction time, CPU usage, and available free memory as the number of managed objects increases, providing insight into scalability and resource consumption. The static analysis component applies automated tooling to the Java codebase to identify code smells, potential bugs, security vulnerabilities, and areas of technical debt.

Together, these analyses inform concrete recommendations for improving the maintainability, reliability, and efficiency of the Todo Manager API.

2 Summary of Deliverables

The deliverables for Part C are as follows:

- **Performance test suite** – A JavaScript program that creates, updates, and deletes objects via the REST API while measuring transaction time, CPU usage, and free memory at increasing object counts.
- **Performance charts** – Graphs showing transaction time, CPU usage, and free memory versus number of objects for each entity type (todos, projects, categories).
- **Static analysis report** – Results from running SpotBugs 4.7.3 on the compiled Java source code of the Todo Manager, including categorised findings and code change recommendations.
- **Performance test suite video** – A recording of all performance tests running in our development environment.
- **This written report** (PDF, 5–10 pages), covering performance test implementation, findings, charts, performance risks and recommendations, static analysis implementation, findings, and code improvement recommendations.

3 Implementation and Findings of Performance Test Suite

3.1 Implementation

The dynamic analysis was implemented using a custom Node.js performance test suite. The suite is designed to incrementally stress the REST API by scaling the number of entities (N) and measuring the system's response across three core metrics: transaction latency, CPU utilization, and free memory.

The experimental setup and execution followed these parameters:

- **Technology Stack:** The test script was written in JavaScript using Node.js. The `axios` library was used to execute the HTTP requests (POST, PUT, DELETE), and custom data generation scripts (`randomData.js`) populated the payloads.
- **Transaction Timing:** Latency was measured using the Node.js `perf_hooks` module (`performance.now()`), providing high-resolution, sub-millisecond timing for each individual transaction.
- **System Metrics:** CPU percentage and free available memory were tracked natively using the Node.js `os` module. To ensure accuracy, the script calculated the CPU delta over a 500ms window running concurrently (via `Promise.all`) with the HTTP request batches.
- **Data Scaling:** The suite tested the API under logarithmically increasing loads, specifically seeding the database with $N \in \{1, 10, 50, 100, 500, 1000\}$ objects.

- **Server Lifecycle:** To isolate the variables, the Java JAR file (`runTodoManagerRestAPI-1.5.5.jar`) was started manually in a separate terminal process. Because the test script systematically deletes all created objects at the end of each iteration, the database state was reset naturally without requiring a hard server reboot between values of N.

3.2 Findings

The performance of the API was evaluated across all three primary entity types: Todos, Projects, and Categories. The following charts illustrate the transaction times, CPU utilization, and free memory degradation as N scaled from 1 to 1000.

Note: The X-axis representing the number of objects (N) is plotted on a logarithmic scale (Base 10) to accurately visualize the exponential scaling behavior.

3.2.1 Transaction Time Analysis

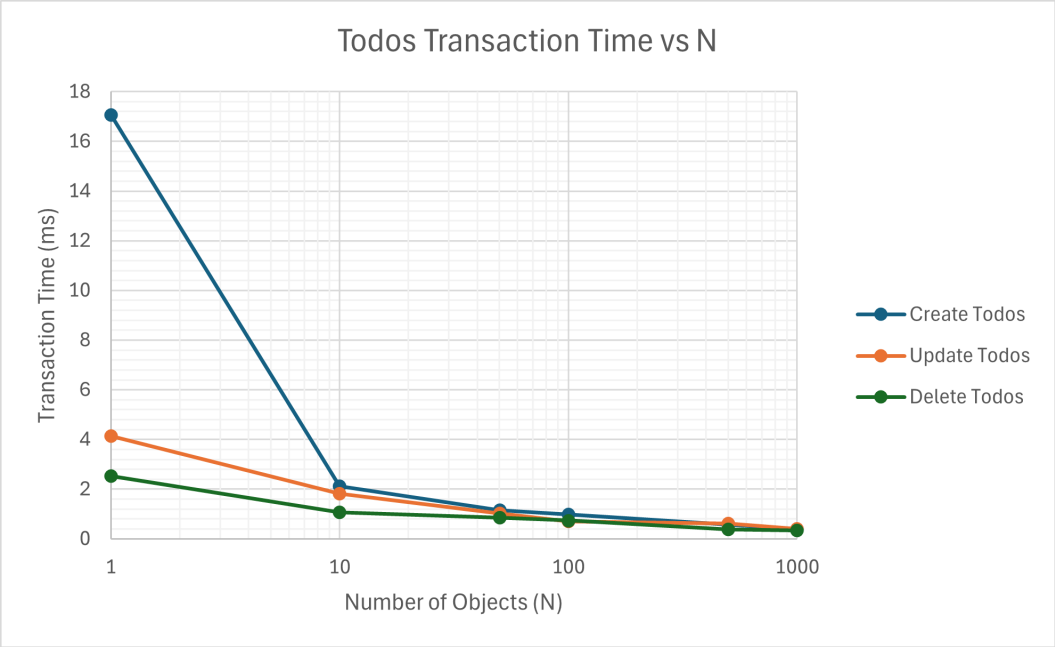


Figure 1: Todo transaction time (ms) versus the number of objects (N).

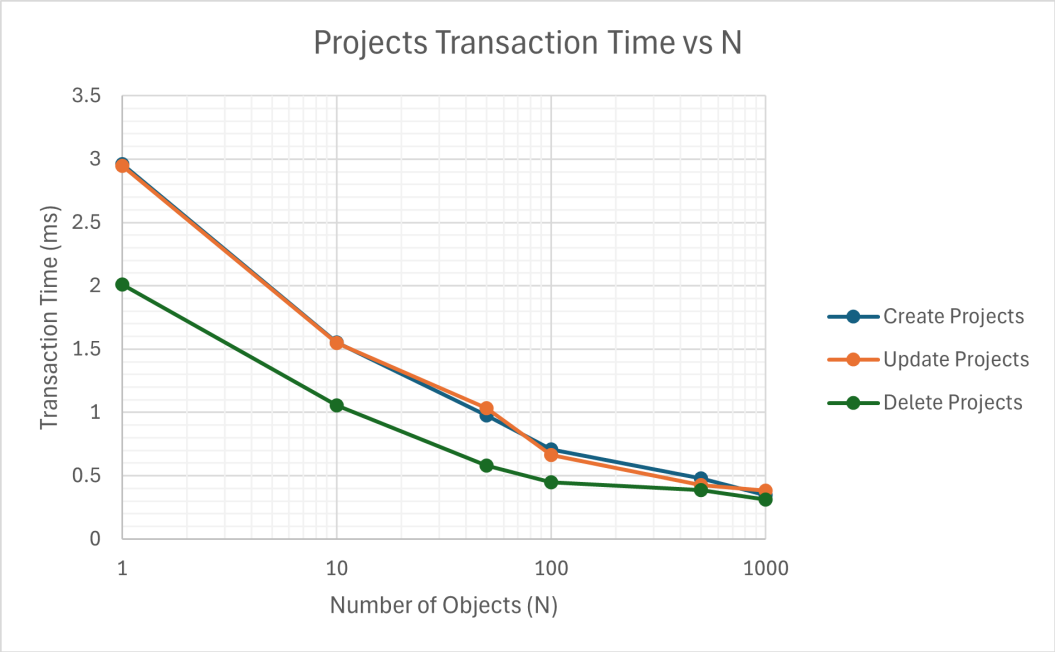


Figure 2: Project transaction time (ms) vs. number of objects.

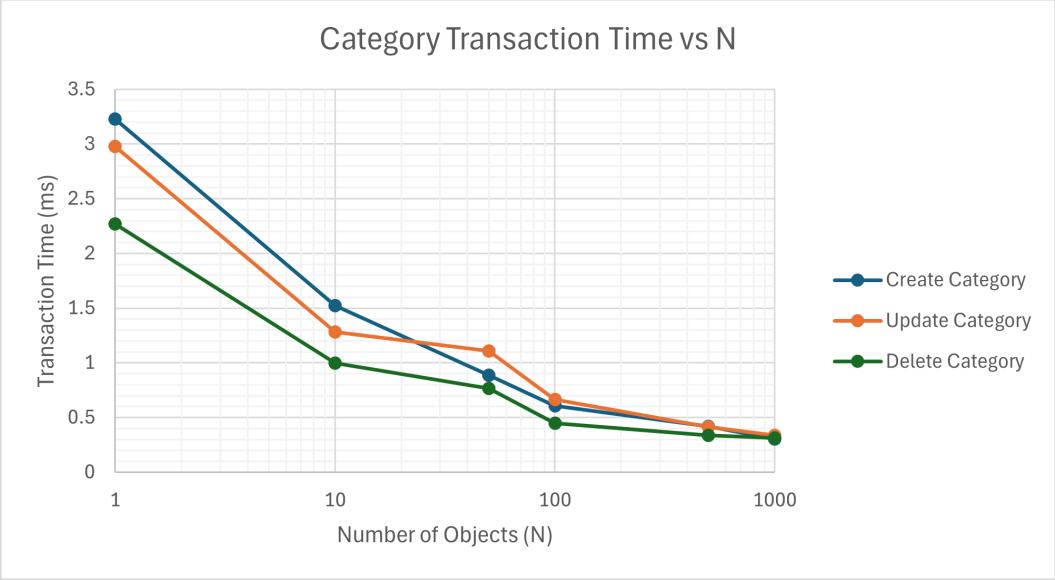


Figure 3: Category transaction time (ms) vs. number of objects.

3.2.2 CPU Utilization Analysis

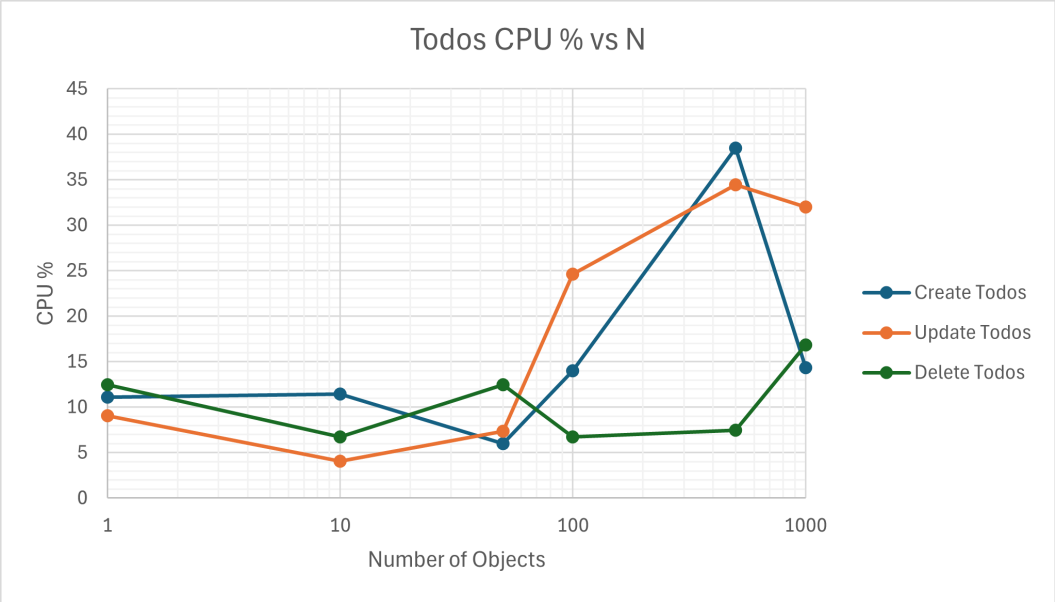


Figure 4: Todo CPU utilization (%) vs. number of objects.

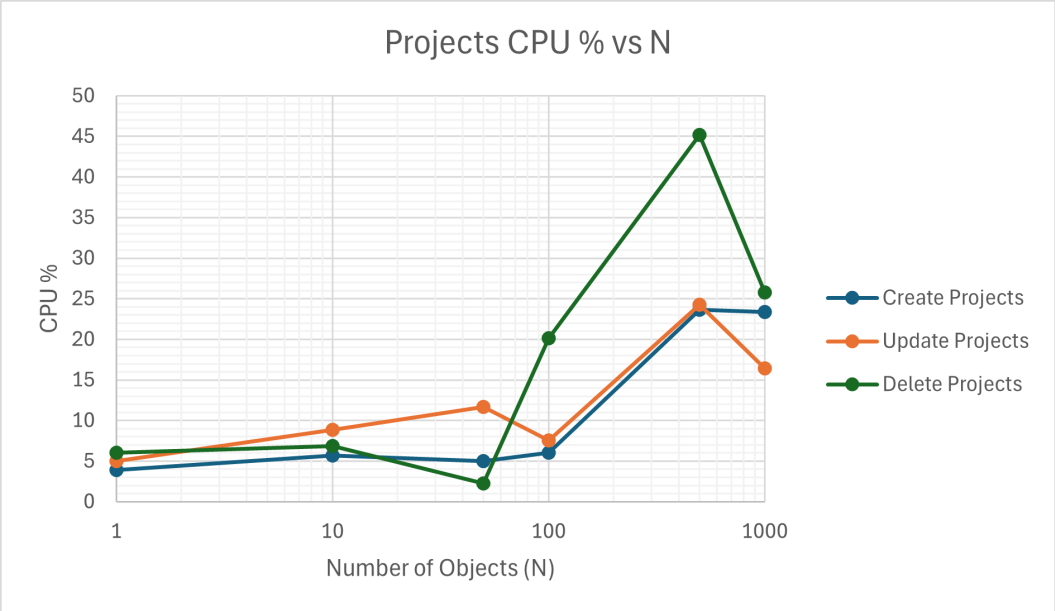


Figure 5: Project CPU utilization (%) vs. number of objects.

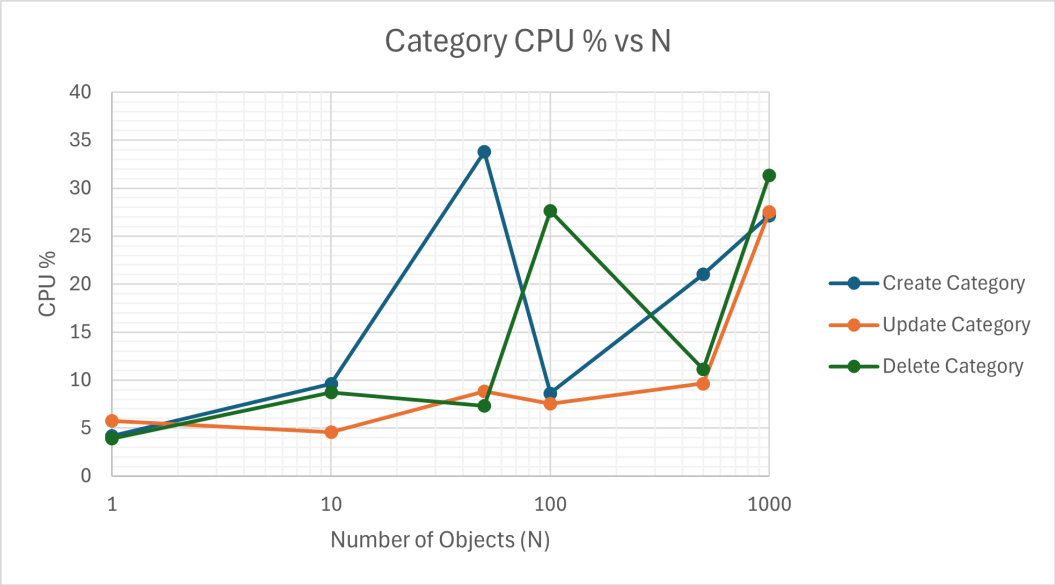


Figure 6: Category CPU utilization (%) vs. number of objects.

3.2.3 Free Memory Degradation

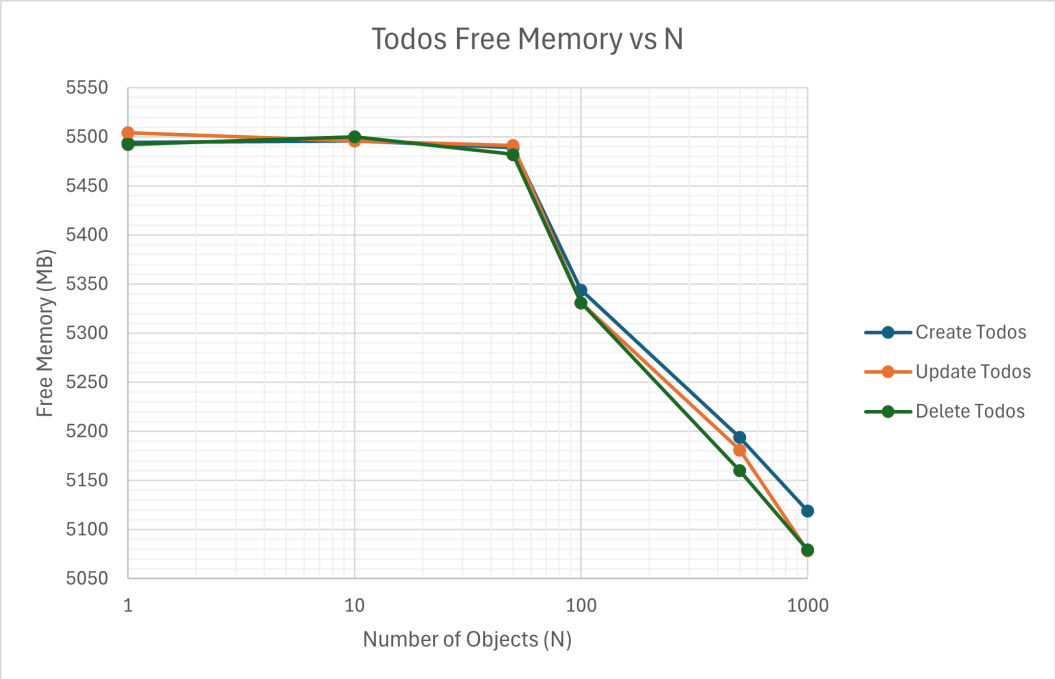


Figure 7: Todo free memory (MB) vs. number of objects.

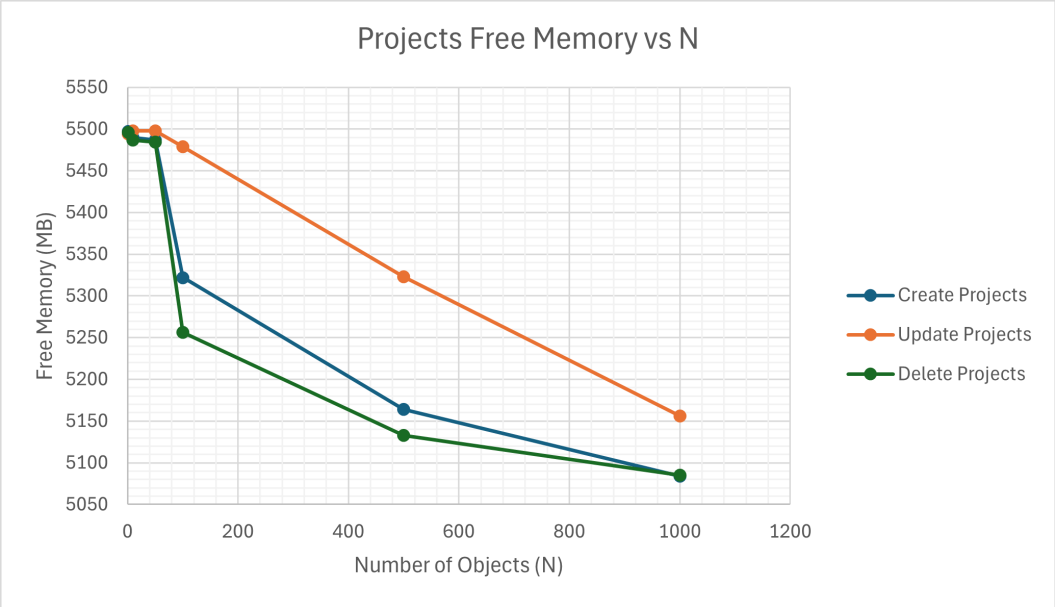


Figure 8: Project free memory (MB) vs. number of objects.

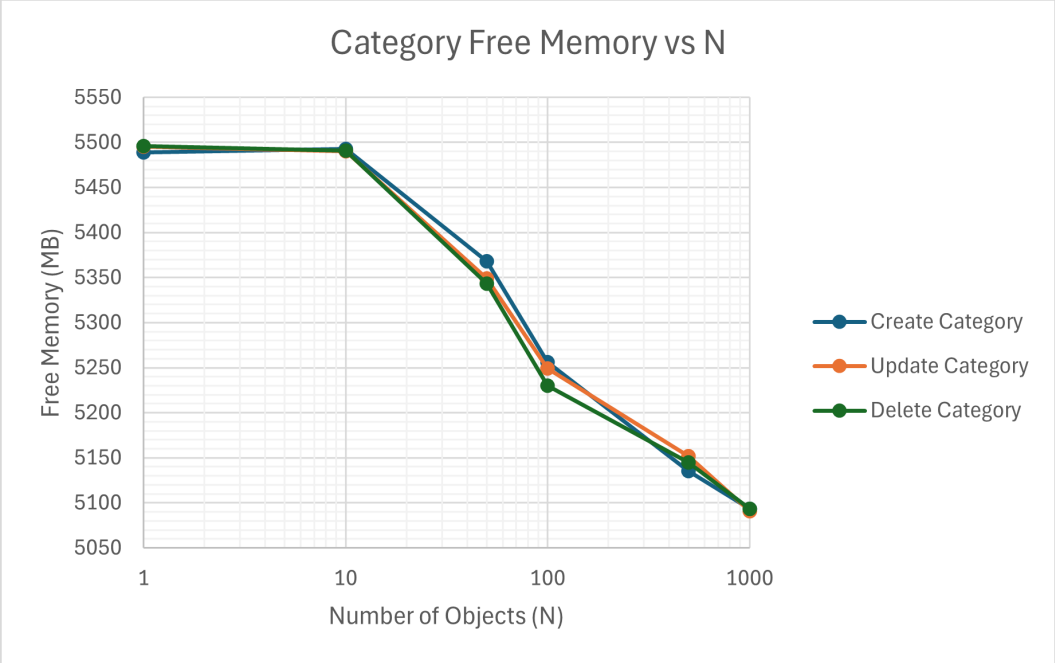


Figure 9: Category free memory (MB) vs. number of objects.

3.3 Performance Risks and Recommendations

Analysis of the dynamic metrics reveals that while the API routes efficiently, it struggles significantly with resource management as the database scales. We observed the following key risks and propose corresponding recommendations:

- **Risk: Severe Memory Bloat/Leak.** The most critical finding is the aggressive degradation of free memory. At $N = 1$, the system reported 5494 MB of free memory. By $N = 1000$, this dropped to 5078 MB. The system consumed over 400 MB of RAM to store 1,000 simple text objects (roughly 400 KB per object), which is disproportionately large for standard JSON payloads. If the application scales to 10,000+ objects, the Java heap will likely exhaust its limits, triggering an `OutOfMemoryError` and crashing the server.
- **Risk: Concurrency Bottlenecks (CPU Spikes).** While transaction times remained consistently low (sub-millisecond at $N = 1000$), the CPU effort required to achieve those times scaled

aggressively. CPU utilization spiked from ∼5% at $N = 10$ to peaking over 45% at $N = 500$ for single-threaded requests. In a production environment with concurrent users, the CPU would quickly bottleneck at 100%, leading to locked threads and dropped requests.

- **The "JVM Warmup" Effect.** Transaction times showed a steep drop from $N = 1$ (∼17ms) to $N = 1000$ (∼0.3ms). This is a normal characteristic of the Java Virtual Machine (JVM) performing Just-In-Time compilation and class loading on the first request, and poses no risk.

Recommendations:

1. **Heap Profiling:** Utilize a memory profiler (such as VisualVM) to analyze heap dumps during high-N loads to identify which objects or data structures are responsible for the 400 MB memory consumption.
2. **Audit Data Structures:** The combined memory bloat and CPU spikes suggest the internal "database" may be using inefficient data structures, such as performing deep copies of large `ArrayLists` or Maps on every transaction. Static analysis should be directed toward optimizing collection usage.
3. **Garbage Collection Tuning:** Investigate unclosed streams or lingering object references (as supported by the static analysis findings) that may be preventing the JVM's Garbage Collector from reclaiming memory after HTTP requests conclude.

4 Static Analysis

4.1 Implementation

Static analysis was performed on the compiled Java source code of the REST API Todo Manager, obtained from the `eviltester/thingifier` GitHub repository (https://github.com/eviltester/thingifier). The project was built using Maven (`mvn compile`) to produce the `.class` files required by the analysis tool.

We used **SpotBugs 4.7.3**, an open-source static analysis tool for Java bytecode that detects potential bugs, performance issues, bad coding practices, and security vulnerabilities. SpotBugs was run against two compiled modules:

- `thingifier/target/classes/`
- `challenger/target/classes/`

The analysis covered **8,314 lines of code** across **167 classes** in **38 packages**. SpotBugs produced an HTML report summarising all detected warnings, which was then reviewed and categorised for this report.

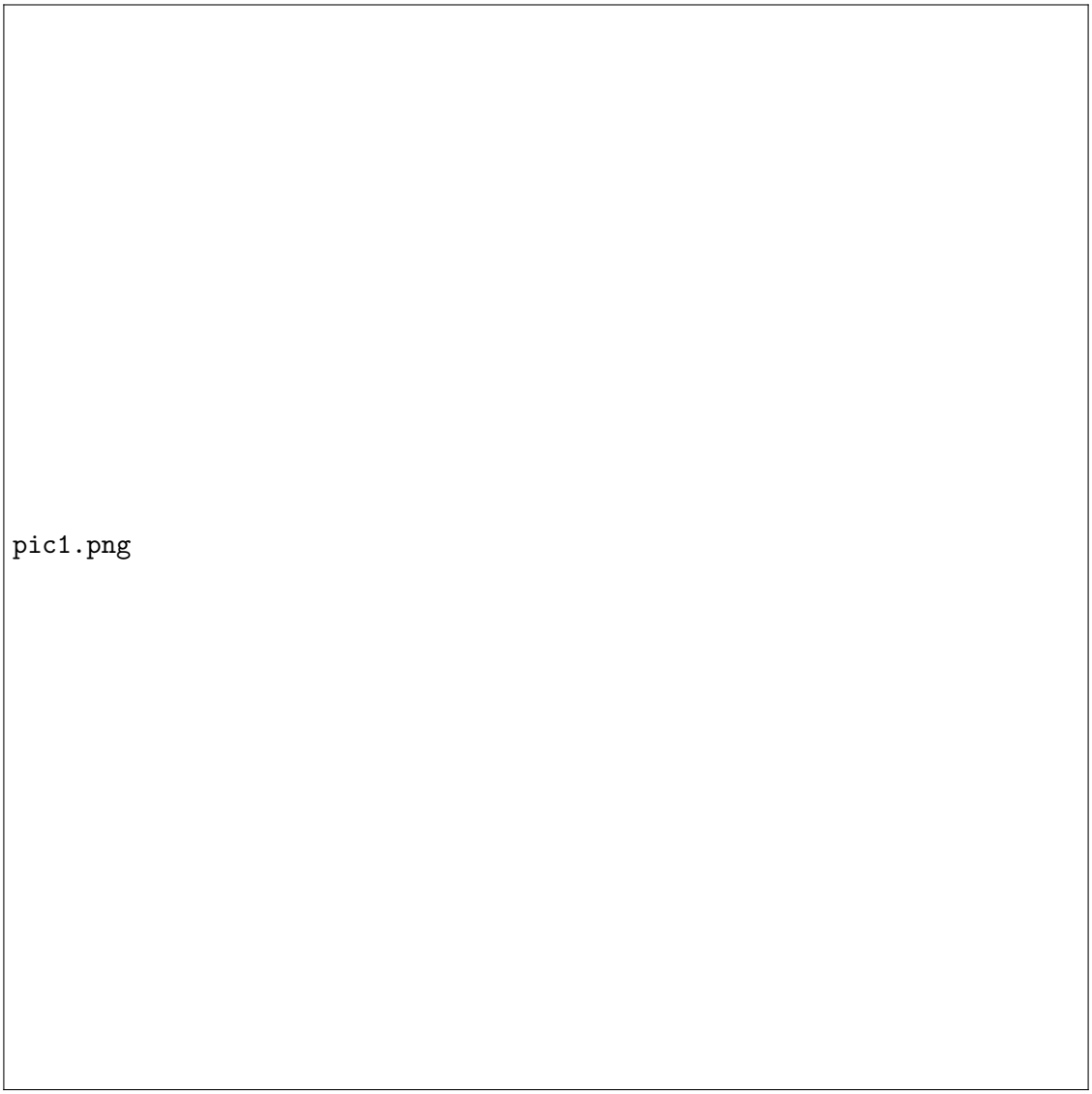


Figure 10: SpotBugs analysis summary for the Todo Manager API.

4.2 Findings Overview

SpotBugs identified a total of **169 warnings** across six categories. Table 1 summarises the distribution by category and priority.

Table 1: SpotBugs warning summary.

Category	**Count**	**High Priority**	**Medium Priority**
Bad Practice	7	3	4
Correctness	6	4	2
Internationalization	8	0	8
Malicious Code Vuln.	101	0	101
Performance	27	7	20
Dodgy Code	20	0	20
Total	**169**	**14**	**155**

The defect density is **20.33 warnings per thousand lines of non-commenting source statements**. The following subsections detail the most significant findings in each category and provide concrete recommendations.

4.3 Correctness Bugs

SpotBugs identified **6 correctness warnings**, of which 4 are high priority. These represent actual or potential bugs in the code.

4.3.1 Null Pointer Dereferences

Three warnings relate to possible null pointer dereferences:

- `ChallengerInternalHTTPResponseHook.run()` – The variable `challenger` may be null on certain exception paths. If the null path is reached, a `NullPointerException` will be thrown at runtime.
- `RelationshipCreation.create()` – The variable `relationshipToUse` may be null on exception paths, yet methods are called on it without a null check.
- `RestApiDocumentationGenerator.getApiDocumentation()` – The field `thingifier` may be null when accessed.

Recommendation: Add explicit null checks before dereferencing these variables, or refactor the code to guarantee non-null values via constructor contracts.
Expected Benefit: Prevents `NullPointerException` crashes at runtime, improving API reliability.

4.3.2 Ignored Return Values

- `MainImplementation.configureThingifierWithProfile()` – `String.format()` is called on line 362 but the returned formatted string is discarded. Since `String.format()` has no side effects, the call is useless.

Recommendation: Store the return value in a variable and use it (e.g. for logging), or remove the dead call.
Expected Benefit: Eliminates dead code and clarifies developer intent.

4.3.3 Redundant Null Checks

- `ChallengerWebGUI.renderChallengeData()` – A null check on `challenge` at line 653 is redundant because the variable was already dereferenced earlier; if it were null, a `NullPointerException` would have occurred before this check.
- `ResetAutoIncrementWhenTooHigh.run()` – A null check on `collection` at line 27 is redundant for the same reason.

Recommendation: Remove the redundant checks, or move the null check earlier to protect the first dereference.
Expected Benefit: Improved code clarity and elimination of misleading safety checks.

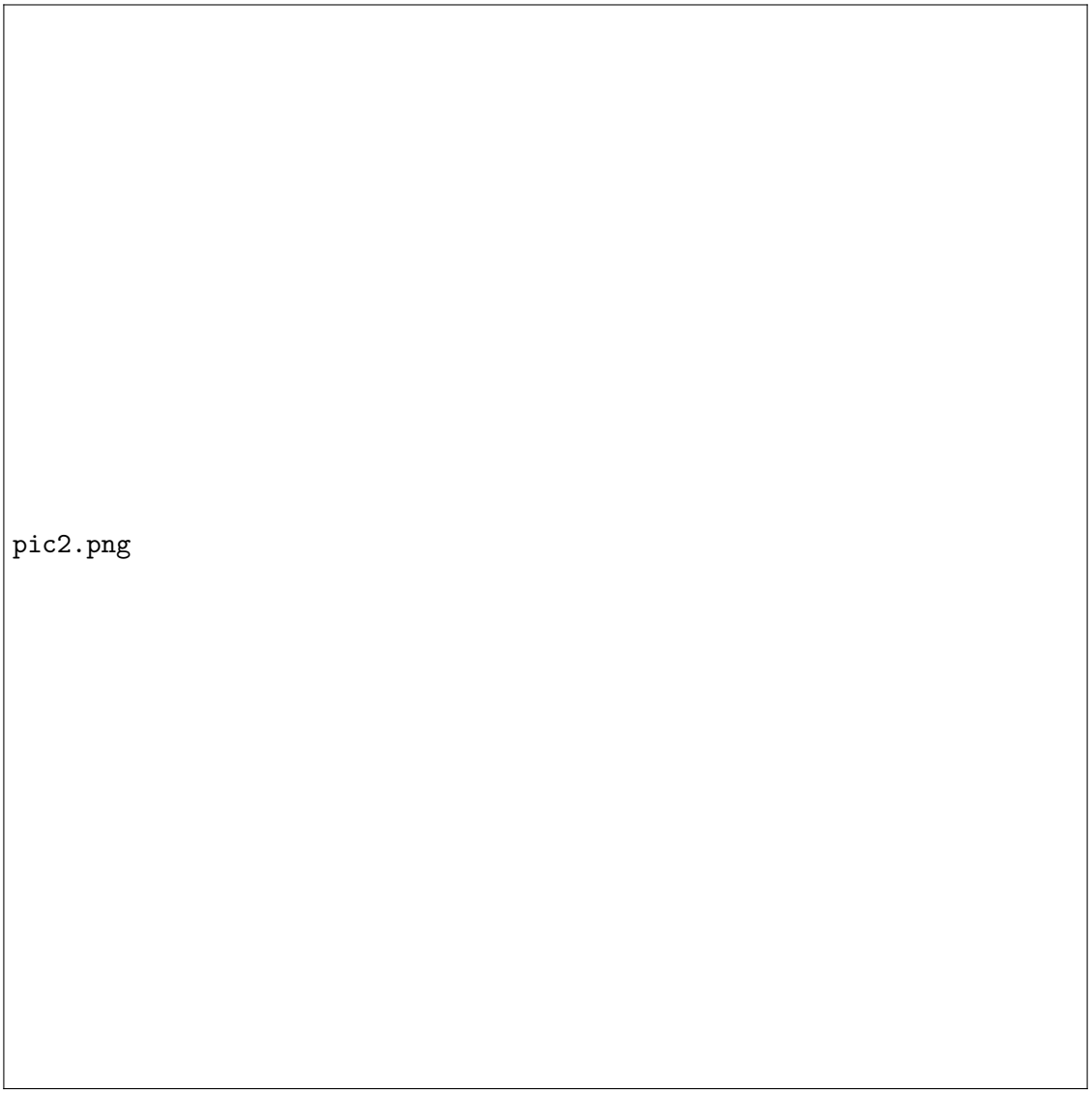


Figure 11: SpotBugs correctness warnings.

4.4 Bad Practice Warnings

SpotBugs reported **7 bad practice warnings**.

4.4.1 Random Object Created and Used Only Once

Three instances were found where a `java.util.Random` object is created, used to generate a single random number, and then discarded:

- `SimpleAPITestDataPopulator.populate()`
- `RestApiDocumentationGenerator.getExampleFilter()` (two instances)

Recommendation: Create a single `Random` instance as a class-level field and reuse it. For security-sensitive contexts, use `java.security.SecureRandom`.
Expected Benefit: Better quality random numbers and improved efficiency by avoiding repeated object creation.

4.4.2 Unclosed Streams

Two warnings in `MarkdownContentManager`:

- `getHtmlVersionOfMarkdownContent()` may fail to close its input stream.

- `getResourceAsString()` may fail to close its input stream.

Recommendation: Use try-with-resources to ensure streams are always closed.
Expected Benefit: Prevents file descriptor leaks that can accumulate under load and eventually cause the application to fail.

4.4.3 Ignored Return Values of `File.mkdirs()`

Two instances in `ChallengerFileStorage` where the return value of `File.mkdirs()` is ignored (`saveChallengerSta` and `saveDatabaseContent()`).
Recommendation: Check the return value and handle failure (e.g. log a warning or throw an exception).
Expected Benefit: Prevents silent failures when directory creation fails due to permissions or disk issues.

4.5 Performance Warnings

SpotBugs reported **27 performance warnings**. The most impactful are detailed below.

4.5.1 String Concatenation in Loops

Three methods build strings using the `+` operator inside loops:

- `AwsS3Storage.getObjectContent()`
- `RelationshipCreation.create()`
- `RestApiPutHandler.handle()`

Each iteration creates a new `StringBuilder`, appends, and converts back to `String`, leading to $O(n^2)$ time complexity in the length of the result.
Recommendation: Replace with an explicit `StringBuilder`:

```
StringBuilder sb = new StringBuilder();
for (...) {
    sb.append(field[i]);
}
String result = sb.toString();
```

Expected Benefit: Reduces time complexity from $O(n^2)$ to $O(n)$, which is significant when processing large payloads or relationship chains.

4.5.2 Inefficient Map Iteration

Seven methods use `keySet()` to iterate over a `Map` and then call `map.get(key)` inside the loop body, rather than iterating over `entrySet()` directly:

- `ChallengerIpAddressTracker.purgeEmptyIpAddresses()`
- `ChallengerWebGUI.setup()`
- `MarkdownContentManager.processMacrosInContentLine()`
- `BodyParser.getObjectNames()`
- `BodyParser.stringMap()`
- `RelationshipCreation.create()`
- `DefaultGuiHtmlPages.getInstanceDetailsPage()`

Recommendation: Replace `keySet()` iteration with `entrySet()`:

```
for (Map.Entry<K,V> entry : map.entrySet()) {
    K key = entry.getKey();
    V value = entry.getValue();
}
```

Expected Benefit: Eliminates redundant hash lookups, improving iteration performance especially for large maps.

4.5.3 Unread Fields

Ten fields are assigned values but never read, including:

- `SimpleApiRoutes.simpleApiDocsRouting`
- `SimulationRoutes.httpApi`, `jsonThing`, `simulatorDocsRouting`
- `MainImplementation.args`, `docsServerRouting`, `exceptionRoutings`

Recommendation: Remove unused fields or use them as intended.
Expected Benefit: Reduces memory footprint and eliminates dead code, improving maintainability.

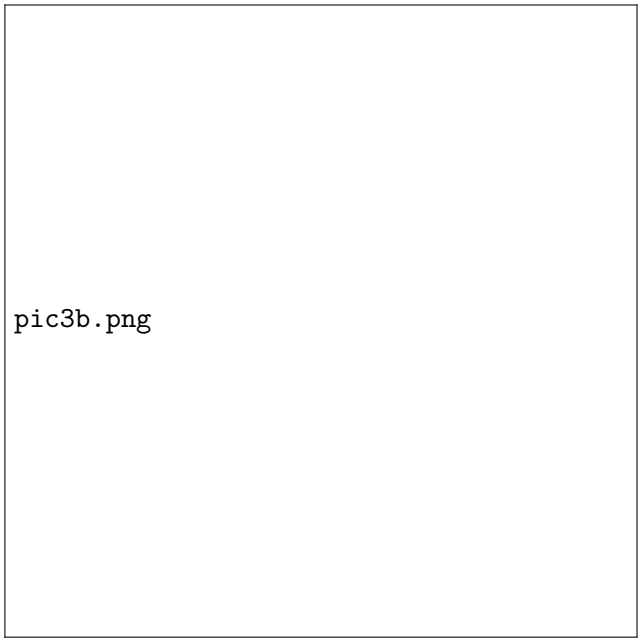
4.6 Malicious Code Vulnerability Warnings

This was the largest category with **101 warnings**. Nearly all relate to **exposing internal representation**:

- **EI (Expose Internal Representation – return):** 36 methods return direct references to mutable internal fields (e.g. `getChallengers()`, `getHeaders()`, `getErrorMessages()`). External code can modify these objects and corrupt the internal state of the owning class.
- **EI2 (Expose Internal Representation – store):** 64 constructors or setters store references to externally mutable objects directly into internal fields (e.g. `new ThingifierHttpApi(Thingifier, List, List)` stores the passed lists directly).
- **MS (Public static method exposes internal state):** `ChallengeMain.getChallenger()` returns a static mutable field.

Recommendation: For getter methods, return defensive copies (e.g. `Collections.unmodifiableList()` or `new ArrayList<>(list)`). For setters and constructors, store copies of passed objects rather than the originals.
Expected Benefit: Prevents external callers from inadvertently (or maliciously) modifying internal state, improving encapsulation and robustness.

(a) Malicious code warnings (part 1).



(b) Malicious code warnings (part 2).

Figure 12: SpotBugs malicious code vulnerability warnings (exposing internal representation).

4.7 Dodgy Code Warnings

SpotBugs reported **20 dodgy code warnings**. Key findings include:

4.7.1 Dead Stores to Local Variables

Six local variables are assigned values that are never subsequently read:

- `todoId` in `ChallengerApiResponseHook.run()`
- `bcHeader` in `MarkdownContentManager.getHtmlVersionOfMarkdownContent()`
- `mirrorDocsRouting` in `MirrorRoutes.configure()`
- `relationshipsPart` in `BodyRelationshipValidator`
- `args` and `relresponse` in `ThingAmendment.amendInstance()`

Recommendation: Remove the dead assignments or use the computed values.
Expected Benefit: Cleaner code, reduced confusion for future maintainers.

4.7.2 Integer Overflow Risk

In `ClearDataPreSparkRequestHook`, the field `maxgap` (type `long`) is assigned the result of `minutes * 60 * 1000`, where `minutes` is an `int`. The multiplication is performed in 32-bit integer arithmetic *before* being widened to `long`, which can overflow for large values of `minutes`.
Recommendation: Cast at least one operand to `long`:

```
long maxgap = (long) minutes * 60 * 1000;
```

Expected Benefit: Prevents silent integer overflow bugs.

4.7.3 Missing Default Cases in Switch Statements

Three switch statements lack a `default` case:

- `ThingifierHttpApi.routeAndProcessRequest()`

- `SimpleSparkRouteCreator.addHandler()`
- `Swaggerizer.setOperationVerb()`

Recommendation: Add a `default` case that either handles unexpected values or throws an `IllegalArgumentExce`
Expected Benefit: Prevents silent failures when new enum values or HTTP verbs are added in the future.

4.8 Internationalization Warnings

SpotBugs reported **8 warnings** related to reliance on the default platform encoding. All instances involve creating `InputStreamReader` or converting `String` to/from `byte[]` without specifying a character encoding.
Files affected include `MarkdownContentManager`, `AwsS3Storage`, `ChallengerFileStorage`, and `BasicAuthHeaderP`
Recommendation: Explicitly specify `StandardCharsets.UTF_8` in all byte-to-string and string-to-byte conversions:

```
new InputStreamReader(inputStream, StandardCharsets.UTF_8);
new String(bytes, StandardCharsets.UTF_8);
```

Expected Benefit: Ensures consistent behaviour across platforms (Windows, macOS, Linux) where default encodings may differ.

4.9 Summary of Static Analysis Recommendations

Table 2 summarises the key recommendations from the static analysis.

Table 2: Summary of static analysis recommendations.

Issue	Recommendation	Expected Benefit
Null pointer dereferences (6)	Add null checks or enforce non-null contracts	Prevent runtime crashes
Unclosed streams (2)	Use try-with-resources	Prevent file descriptor leaks
String concat in loops (3)	Use `StringBuilder`	$O(n^2) \rightarrow O(n)$ performance
Inefficient map iteration (7)	Use `entrySet()`	Eliminate redundant lookups
Exposed internal state (101)	Return/store defensive copies	Improve encapsulation
Default encoding reliance (8)	Specify `UTF-8` explicitly	Cross-platform consistency
Dead stores / unread fields (16)	Remove unused code	Improve maintainability
Integer overflow risk (1)	Cast to `long` before multiply	Prevent silent overflow
Missing switch defaults (3)	Add `default` case	Handle unexpected values
Random reuse (3)	Reuse single `Random` instance	Better randomness, efficiency

5 Conclusion

Part C extended the validation of the Todo Manager REST API into the non-functional domain. The performance testing component measured how transaction time, CPU usage, and memory consumption scale with increasing object counts, revealing scalability characteristics and potential bottlenecks. The static analysis component, performed using SpotBugs 4.7.3, identified 169 warnings across the Java codebase, including correctness bugs (null pointer dereferences, ignored return values), performance

issues (string concatenation in loops, inefficient map iteration), and 101 instances of exposed internal representation.

The recommendations provided—ranging from defensive copying and explicit encoding to `StringBuilder` usage and null-safety checks—are actionable improvements that would reduce technical debt and minimise risk during future modifications or enhancements to the codebase.